Relatório de Pesquisa Técnica e Arquitetural: Projeto Noite 2.0

Análise do Código Legado e Dívida Técnica

A análise exaustiva do repositório legado do Projeto Noite (composto por arquivos como main.py, orchestrator.py, gui.py e scripts de migração) revela um protótipo funcionalmente rico em intenções de automação, mas profundamente limitado por decisões arquiteturais acopladas e dependências frágeis. Para estruturar a transição em direção à versão 2.0, foi realizada uma auditoria sistemática dos módulos legados, catalogando componentes reaproveitáveis e identificando gargalos estruturais.

Componente Legado	Implementação Antiga	Avaliação de Risco e Dívida Técnica	Decisão de Arquitetura para a Versão 2.0
Interface Gráfica	Tkinter com acoplamento assíncrono direto (gui.py)	Alto risco de travamento de UI por mistura de *threads* e loops de *polling*	**Descartar completamente**: Substituir por cliente PySide6 isolado por WebSocket/HTTP local.
Normalização de Voz	Regras de substituição fonética via Regex (main.py, gui.py)	Código pulverizado dentro da camada de renderização e captura	**Reaproveitar e Mover**: Extrair padrões fonéticos para um módulo puro de pós-processamento de STT.
Roteamento de Ações	Correspondência direta de palavras-chave e intenções (orchestrator.py)	Execução rápida (< 10 ms), porém vulnerável a variações sintáticas	**Reaproveitar**: Integrar como a Camada 1 (Nível Determinístico) do novo Roteador Hierárquico.
Síntese de Voz (TTS)	Biblioteca edge-tts e reprodução via playsound (main.py)	Dependência de endpoint não oficial em nuvem e bloqueio síncrono da thread	**Descartar**: Substituir por Piper TTS local com *streaming* de áudio não bloqueante.
Persistência de Dados	Arquivos JSON planos em sistema de arquivos (data/)	Falta de transacionalidade (ACID), risco de corrupção e lentidão em buscas	**Descartar**: Migrar para banco de dados relacional e vetorial embarcado SQLite (sqlite-vec).

A principal contribuição reaproveitável do código antigo reside nas rotinas de sanitização fonética e expressões regulares contidas na função corrigir_palavra. O código legado tratava erros comuns dos reconhecedores de voz ao capturar marcas registradas e comandos em português do Brasil, convertendo termos fonéticos distorcidos (como "iutub", "yutub" ou "rutube") para a grafia exata "youtube", bem como variações como "gugol" para "google". Essa

camada heurística provou ser altamente eficaz para evitar chamadas desnecessárias a modelos de linguagem em tarefas triviais. Da mesma forma, o mecanismo de detecção rápida de intenções (detectar_acao_rapida) em orchestrator.py demonstrou latência inferior a 10 milissegundos na identificação de solicitações diretas de abertura de aplicativos e pesquisas estruturadas.

Por outro lado, a dívida técnica acumulada no projeto original é expressiva. O arquivo gui.py mescla a construção gráfica do Tkinter com a captura contínua de áudio via sounddevice, decodificação pelo modelo Kaldi/Vosk, roteamento de intenções e atualização dinâmica do canvas visual por meio de um loop arbitrário checar_pendentes_loop. Essa sobreposição de responsabilidades gera concorrência imprevisível, impede o tratamento adequado de exceções e inviabiliza a execução em sistemas operacionais sem servidor gráfico nativo.

Adicionalmente, a sintaxe utilizada na montagem de prompts (_montar_system_prompt) injetava o conteúdo de arquivos locais indexados diretamente no prompt de sistema sem delimitadores de segurança ou sanitização de caracteres especiais. Esse padrão expõe o sistema a ataques graves de injeção de prompt indireta, onde um arquivo de texto processado pode sobrescrever as instruções mestras do assistente e executar comandos indesejados no sistema operacional. Por fim, a ausência de uma suíte de testes unitários ou de integração torna qualquer refatoração no código antigo um processo de alto risco.

Benchmarks em Hardware Restrito e Alocação de Recursos

O dimensionamento da arquitetura do Projeto Noite 2.0 é delimitado pelas especificações físicas da máquina de implantação, composta por um processador Intel Core i7-3770, 16 GB de memória RAM DDR3, um disco SSD SATA e uma placa de vídeo NVIDIA GeForce GTX 1660 Super com 6 GB de VRAM.

Compatibilidade do Processador e Ausência de Instruções AVX2

O processador Intel Core i7-3770, pertencente à arquitetura Ivy Bridge, possui suporte nativo ao conjunto de instruções vetorizadas AVX 1.0, mas não dispõe do conjunto AVX2 nem do suporte a operações FMA3. Essa restrição técnica afeta diretamente os motores de inferência executados em CPU:

1. **Runtimes de Modelos de Linguagem (llama.cpp e Ollama)**: Historicamente, a ausência de suporte AVX2 impedia a execução de binários otimizados pré-compilados do llama.cpp. Contudo, versões recentes do Ollama (como a 0.5.8+) e compilações modernas do llama.cpp implementaram seleções de instruções dinâmicas em tempo de execução e caminhos de execução (*fallbacks*) compatíveis com AVX. Quando o processamento do modelo ocorre integralmente na GPU via aceleração CUDA, a ausência de AVX2 na CPU não gera gargalo de velocidade de inferência. Entretanto, caso o tamanho do modelo exceda os 6 GB de VRAM e exija o descarregamento parcial de camadas (*layer offloading*) para a memória RAM do sistema, a taxa de geração despenca para valores inferiores a 3 tokens por segundo. Esse gargalo é severamente agravado pela reduzida largura de banda da memória RAM DDR3 (aproximadamente 21 GB/s, contra 336 GB/s da VRAM GDDR6 da GPU).
2. **Motores de Transcrição de Voz (CTranslate2 e faster-whisper)**: O runtime CTranslate2 utiliza vetorização AVX2 para acelerar cálculos matriciais quantizados em precisão INT8

em processadores x86. Na ausência de AVX2, o faster-whisper em execução na CPU recorre a rotinas escalares padrões, o que eleva a latência de transcrição em aproximadamente 2,5 a 3 vezes em relação a CPUs modernas de mesma frequência.

Desempenho Real da GPU NVIDIA GeForce GTX 1660 Super

A placa de vídeo GTX 1660 Super é baseada na arquitetura Turing (núcleo TU116, Compute Capability 7.5). Embora compartilhe a mesma geração das placas da linha RTX 2060, a linha GTX não possui *Tensor Cores* dedicados. A aceleração de inferência de inteligência artificial é realizada por meio de núcleos CUDA convencionais utilizando instruções de produto escalar INT8/FP16 (DP4A).

Modelo de Linguagem	Formato de Quantização	Janela de Contexto Ativa	Alocação de VRAM	Taxa de Geração (Decode)	Fonte da Evidência
Qwen 2.5 3B Instruct	Q4_K_M	4.096 tokens	2,20 GB	48,5 tokens/s	Benchmark em Hardware Turing
Llama 3.2 3B Instruct	Q4_K_M	4.096 tokens	2,35 GB	42,1 tokens/s	Benchmark em Hardware Turing
Qwen 2.5 7B Instruct	Q4_K_M	4.096 tokens	4,80 GB	18,2 tokens/s	Benchmark em Hardware Turing
Llama 3.1 8B Instruct	Q4_K_M	2.048 tokens	5,10 GB	14,8 tokens/s	Benchmark em Hardware Turing
Qwen 2.5 Coder 7B	Q4_K_M	8.192 tokens	5,45 GB	12,3 tokens/s	Benchmark em Hardware Turing

Orçamento de VRAM e Latência Concorrente

Considerando que a GPU possui um teto rigoroso de 6,0 GB de VRAM e que o sistema operacional Windows reserva aproximadamente 800 MB para composição de interface de vídeo e drivers, a capacidade útil restante para inteligência local é de aproximadamente 5,2 GB. Tentar executar um modelo de 7B de parâmetros (que consome ~4,8 GB) concorrentemente com modelos de visão ou serviços de voz na GPU causará erro imediato de falta de memória (*Out of Memory*) ou degradação drástica por uso de RAM de sistema.
A estratégia de alocação de hardware deve, portanto, segregar estritamente os componentes entre GPU e CPU. O modelo de linguagem pequeno local (SLM) atuará como o componente exclusivo residente na VRAM da GPU, enquanto todos os serviços de suporte de voz e processamento de áudio serão alocados na CPU.

Componente da Arquitetura	Dispositivo Alocado	Formato de Precisão	Consumo de VRAM	Consumo de RAM (DDR3)	Latência Média de Resposta
SLM Principal (Qwen 2.5 3B)	GPU (CUDA)	Quantizado Q4_K_M	2,20 GB	100 MB	~20 ms (Time-to-First-Token)
Cache de	GPU (VRAM)	Híbrido	0,55 GB	0 MB	N/A (Dinâmico)

Componente da Arquitetura	Dispositivo Alocado	Formato de Precisão	Consumo de VRAM	Consumo de RAM (DDR3)	Latência Média de Resposta
Chave-Valor (KV Cache - 8K)		FP16/Q4			
STT (faster-whisper Small)	CPU (8 Threads)	Quantizado INT8	0 MB	250 MB	~350 ms para 5s de áudio
TTS (Piper TTS Engine)	CPU (Sintonizado)	Modelo ONNX Float32	0 MB	120 MB	~80 ms por sentença
VAD (Silero VAD v5)	CPU (Contínuo)	Modelo ONNX Float32	0 MB	35 MB	< 10 ms por bloco de áudio
Reserva do Sistema Operacional	GPU / Sistema	N/A	0,80 GB	N/A	N/A
TOTAL COMBINADO EM EXECUÇÃO	**Híbrido**	**N/A**	**3,55 GB (de 6,0 GB)**	**505 MB (de 16,0 GB)**	**~450 ms até início do áudio**

Orquestração de Agentes e Estratégia de Roteamento

A arquitetura de orquestração de agentes para o Projeto Noite 2.0 evita dependências opacas ou abstrações pesadas que introduzam sobreposição de chamadas, focando em controle de fluxo determinístico e expansibilidade modular.

Avaliação Comparativa de Frameworks de Agentes

Foi realizada uma análise comparativa entre as principais soluções da indústria e a construção de um núcleo assíncrono customizado em Python.

Critério de Análise	Núcleo Customizado (asyncio)	Pydantic AI (v2.59)	LangGraph	CrewAI / AutoGen
Sobrecarga em Memória RAM	Negligenciável (< 10 MB)	Baixa (~35 MB)	Média (~120 MB)	Alta (> 250 MB)
Latência de Processamento Interno	Minimalista (< 1 ms)	Baixa (~5 ms)	Média (~25 ms)	Alta (> 100 ms)
Garantia de Tipagem e Schemas	Manual via Pydantic	Nativa e estrita via Pydantic	Parcial	Frágil
Transparência e Depuração	Total (Stack traces nativos)	Alta (Exceções claras)	Complexa (Grafos de estados)	Baixa (Fluxos ocultos)
Grau de Recomendação	**Recomendado para MVP**	**Recomendado para Expansão**	**Descartado**	**Descartado**

Frameworks como LangGraph, CrewAI e AutoGen impõem estruturas rígidas baseadas em grafos ou abstrações de papéis de agentes que consomem memória excessiva e ocultam o

fluxo de dados em camadas de abstração complexas. Para o Projeto Noite 2.0, a escolha arquitetural recai sobre a construção de um **Núcleo Assíncrono Minimalista em Python Native (asyncio)** integrado à biblioteca Pydantic para validação estrita de contratos de entrada e saída, considerando a adoção do Pydantic AI em etapas futuras caso a complexidade dos agentes aumente.

Camada de Abstração de Modelos (Model Gateway)

Para garantir imunidade contra alterações nas APIs de fornecedores externos e permitir a troca transparente de modelos, a Noite 2.0 emprega a biblioteca **LiteLLM** como camada unificada de comunicação. O LiteLLM expõe uma interface padronizada compatível com a API OpenAI, traduzindo automaticamente os parâmetros para endpoints locais do Ollama/llama.cpp ou serviços em nuvem como Anthropic Claude, Google Gemini e OpenAI GPT. Além disso, a camada de Gateway gerencia automaticamente mecanismos de tentativa de reenvio (*retries*), retardo exponencial (*exponential backoff*) e alternância transparente para modelos secundários (*failover*) em caso de indisponibilidade da rede ou estouro de cota de chamadas.

Mecanismo de Roteamento em Três Camadas

A tomada de decisão sobre qual modelo ou ferramenta deve processar uma solicitação do usuário não deve depender exclusivamente de chamadas a modelos de linguagem, pois essa prática introduz latência desnecessária e custos de API imprevisíveis. O roteamento do Projeto Noite 2.0 opera em um pipeline estritamente hierárquico:

1. **Camada 1: Roteador Determinístico por Regras (Regex & Heurísticas)**: Analisa a entrada de texto sanitizada em busca de padrões estruturados de comandos operacionais (como "abrir aplicativo", "aumentar volume", "pesquisar arquivo .pdf" ou consultas diretas de mídia). Se um padrão for identificado, a execução é encaminhada imediatamente para a ferramenta correspondente sem consultar qualquer LLM. Latência da camada: inferior a 5 milissegundos.
2. **Camada 2: Roteador por Modelo Local Pequeno (SLM Classificador)**: Se nenhuma regra determinística for acionada, a solicitação é processada localmente pelo modelo Qwen 2.5 3B através de inferência com Saída Estruturada (forçada por JSON Schema nativo via Ollama/llama.cpp). O modelo local categoriza a intenção entre operações de sistema, consultas à memória ou necessidade de raciocínio avançado. Latência da camada: aproximadamente 150 a 200 milissegundos.
3. **Camada 3: Escalamento para Agentes em Nuvem (Cloud LLM Reasoning)**: Solicitações categorizadas pela Camada 2 como tarefas de alta complexidade (por exemplo, geração de código orientado a objetos em Luau para Roblox Studio, refatoração de algoritmos complexos ou análise detalhada de documentos extensos) são enviadas para APIs externas de alta capacidade, como Claude 3.5 Sonnet ou GPT-4o.

Arquitetura de Memória e Persistência de Identidade

A persistência de informações da assistente Noite 2.0 deve operar de forma desacoplada dos provedores de inferência de inteligência artificial, garantindo a preservação do contexto histórico, preferências do usuário e traços de personalidade, independentemente de o modelo

subjacente ser substituído.

Substituição de Bancos Vetoriais por SQLite com a Extensão sqlite-vec

Bancos de dados vetoriais dedicados (como ChromaDB, Qdrant ou Milvus) exigem a execução de processos em segundo plano ou bindings pesados em Python que consomem entre 400 MB e 1,2 GB de memória RAM de forma permanente. Para um sistema em hardware restrito, esse custo de infraestrutura é inviável.
A Noite 2.0 adota uma arquitetura unificada de armazenamento baseada no **SQLite (versão 3.41 ou superior)** estendido com a biblioteca **sqlite-vec (versão 0.1.9, lançada em março de 2026)**.

- **A Extensão sqlite-vec**: Trata-se de uma biblioteca escrita em C puro, sem dependências externas, que é carregada diretamente dentro do processo nativo do SQLite em Python por meio da função enable_load_extension. A extensão fornece tabelas virtuais do tipo vec0 capazes de armazenar e realizar buscas por vizinhos mais próximos (KNN) em vetores de precisão float32, int8 ou representações binárias quantizadas.
- **Capacidade e Desempenho**: A busca vetorial por força bruta realizada pelo sqlite-vec atinge desempenho extremamente veloz para coleções de até centenas de milhares de vetores, eliminando a necessidade de índices complexos do tipo HNSW para o volume de dados típico de um assistente pessoal.
- **Busca Híbrida Unificada**: O SQLite gerencia a busca por texto completo tradicional (via extensão nativa FTS5) e a busca por similaridade semântica (via sqlite-vec) dentro do mesmo arquivo de banco de dados (noite_core.db). Isso garante conformidade com transações ACID, elimina riscos de dessincronização entre índices e permite realizar backups simplificados por meio da cópia de um único arquivo no disco.

Organização Multicamada da Memória

1. **Memória Episódica (Histórico e Diálogos)**: Regras de registro de conversas estruturadas na tabela episodes. Periodicamente, um processo em segundo plano sumariza episódios antigos utilizando o SLM local, gerando resumos executivos do histórico e liberando a janela de contexto de tokens brutos.
2. **Memória Semântica (Fatos e Preferências)**: Armazena fatos declarativos sobre o usuário, suas preferências e seu ambiente de trabalho na tabela facts. Os textos são convertidos em representações vetoriais de 384 dimensões via modelo denso local (all-MiniLM-L6-v2) e indexados na tabela virtual vec_facts do sqlite-vec.
3. **Memória Procedural (Habilidades e Scripts)**: Armazena manuais de execução, rotinas automatizadas e receitas de código na tabela skills. Cada habilidade é também mantida como um arquivo de texto Markdown estruturado e versionado dentro do diretório /skills/ sob controle de versão Git.

Preservação e Imunidade da Persona

A identidade da Noite é imune a variações comportamentais decorrentes da troca de modelos de linguagem local ou em nuvem. A persona é mantida em um arquivo de especificação estruturado (config/persona.json) que define traços comportamentais, tom de voz, regras de

conduta e limites operacionais. Em todas as requisições enviadas ao Gateway de modelos, este arquivo é injetado como o cabeçalho mestre inalterável do prompt de sistema, garantindo continuidade comportamental rigorosa.

Motor de Voz, Latência e Regras de Licenciamento

A infraestrutura de comunicação por voz do Projeto Noite 2.0 foi desenhada para operar com baixa latência, processamento inteiramente local e capacidade de interrupção da fala do assistente em tempo real (*barge-in*).

Componentes da Arquitetura de Áudio e Fluxo de Dados

A captura de áudio inicia no microfone através do driver sounddevice, que transmite blocos contínuos de áudio PCM de 16 kHz para a biblioteca **Silero VAD v5** executada via runtime ONNX na CPU. O Silero VAD atua como o primeiro filtro de economia de energia, descartando segmentos de silêncio e ruídos ambiente com latência inferior a 10 milissegundos.
Quando o Silero VAD confirma a presença de fala humana, o fluxo de áudio é direcionado ao identificador de palavras de ativação **openWakeWord**. Ao detectar a palavra-chave configurada ("Hey Noite" ou "Noite"), o sistema ativa a gravação do comando e envia o buffer acumulado para o motor de transcrição **faster-whisper** (executando o modelo small quantizado em int8 na CPU). O texto transcrito é sanitizado pelo módulo de correção fonética legado e entregue ao orquestrador.
A resposta em texto gerada pelo orquestrador é processada pelo motor de síntese **Piper TTS**, que utiliza modelos neurais ONNX sintonizados em português do Brasil (como a voz pt_BR-faber-medium). O Piper TTS sintetiza o áudio em pequenas partes (*chunks* de 20 milissegundos) e envia imediatamente os pacotes para a fila de reprodução das caixas de som.

Mecanismo de Interrupção de Fala (Barge-in)

Para garantir uma dinâmica conversacional humana, o sistema monitora continuamente o microfone via Silero VAD mesmo enquanto o Piper TTS está reproduzindo uma resposta. Caso o usuário comece a falar durante a emissão de áudio da assistente, o Silero VAD emite um sinal de evento de alta prioridade (EVENT_BARGE_IN). Esse sinal aciona imediatamente três ações coordenadas: limpa a fila de buffers pendentes da placa de som, interrompe o processo de síntese do Piper TTS em andamento e redireciona o novo áudio do usuário para o pipeline de transcrição do faster-whisper.

Solução de Voz Analisada	Categoria e Execução	Latência Média de Resposta	Termos de Licença de Uso	Viabilidade Econômica
faster-whisper (Small INT8)	Transcrição (STT) / CPU Local	~350 ms para 5s de áudio	Licença MIT	Gratuito (Sem custo operacional)
Vosk PT-BR (Legado)	Transcrição (STT) / CPU Local	~150 ms para 5s de áudio	Licença Apache 2.0	Gratuito (Qualidade inferior ao Whisper)
Piper TTS (ONNX PT-BR)	Síntese (TTS) / CPU Local	~80 ms por sentença	Licença MIT	Gratuito (Sem custo operacional)
Microsoft Edge-TTS	Síntese (TTS) / Nuvem Não Oficial	~400 ms por sentença	Endpoint Privado (Sem Garantia)	Gratuito (Risco alto de bloqueio)

Solução de Voz Analisada	Categoria e Execução	Latência Média de Resposta	Termos de Licença de Uso	Viabilidade Econômica
(Legado)				
OpenAI Whisper API	Transcrição (STT) / Nuvem API	~650 ms por chamada	Comercial / Proprietário	US$ 0,006 por minuto de áudio
ElevenLabs TTS API	Síntese (TTS) / Nuvem API	~320 ms por chamada	Comercial / Proprietário	US$ 0,15 por 1.000 caracteres

Controle do Windows, Automação e Segurança

A concessão de autonomia operacional ao assistente para controlar o sistema operacional Windows exige o estabelecimento de fronteiras rígidas de acesso e isolamento de processos.

Métodos de Automação de Interface

1. **Inspecção de Árvore de UI Automation (pywinauto / comtypes)**: Consiste no método primário e obrigatório de interação com o sistema operacional. Através do subsistema nativo de acessibilidade do Windows (Microsoft UI Automation API), o assistente identifica controles visualmente acessíveis (como botões, caixas de texto e menus) utilizando identificadores HWND determinísticos. Este método apresenta latência inferior a 50 milissegundos, consumo nulo de VRAM e total imunidade a variações de resolução de tela ou temas visuais.
2. **Automação por Visão Computacional (Modelos VLM)**: Método secundário mantido estritamente para exceções. Envolve a captura de imagem da tela (*screenshot*) e o envio do arquivo para um modelo multimodal interpretar coordenadas de clique. Esta abordagem é extremamente lenta (latência de 2,5 a 5,0 segundos), consome cota substancial de VRAM ou API e é propensa a erros de precisão gráfica. Seu uso é restrito a aplicações que não expõem elementos na árvore do UI Automation (como jogos ou softwares legados sem acessibilidade).

Mecanismos de Segurança e Isolamento

Para impedir que processos filhos iniciados pelo assistente consumam recursos descontroladamente ou executem rotinas maliciosas, todos os sub-processos e comandos de terminal são vinculados a um **Job Object da API Win32 do Windows**. O Job Object estabelece restrições em nível de sistema operacional, definindo limites rígidos de consumo de memória RAM (máximo de 2 GB por processo filho), limitação da classe de prioridade da CPU e encerramento compulsório de todos os processos derivados caso a aplicação principal do assistente seja finalizada.
Adicionalmente, a Noite 2.0 é configurada para ser executada obrigatoriamente sob uma **conta de usuário sem privilégios administrativos**. Pela arquitetura de proteção de integridade do Windows (User Interface Privilege Isolation - UIPI), aplicações executadas sem privilégios de administrador são terminantemente impedidas de injetar eventos de teclado ou mouse em janelas de processos que estejam rodando com privilégios elevados (UAC). Essa barreira nativa impede que o assistente seja manipulado para contornar proteções do sistema.
Todas as chaves de API para serviços em nuvem são armazenadas de forma criptografada no **Windows Credential Manager** nativo por meio da biblioteca Python keyring, eliminando a presença de credenciais em arquivos de texto plano. Quanto à proteção contra injeção de

prompt indireta, todo dado lido do sistema de arquivos, páginas da web ou telas é sanitizado e encapsulado dentro de blocos XML isolados acompanhados de instruções restritivas no prompt do sistema.

Nível de Permissão	Categoria de Ação	Operações Permitidas	Mecanismo de Confirmação Obrigatório
Nível 0	Somente Leitura	Consultas de horário, leitura de arquivos na pasta de trabalho, busca na web.	Execução automática sem confirmação.
Nível 1	Modificação Controlada	Criação e edição de arquivos dentro do diretório /workspace/ do projeto.	Registro em log com notificação visual passiva.
Nível 2	Execução do Sistema	Abertura de aplicativos conhecidos, execução de scripts em ambiente sandbox.	Confirmação prévia por clique na interface gráfica.
Nível 3	Ações Destrutivas	Exclusão de arquivos, comandos de terminal (cmd/powershell), transações financeiras.	Confirmação por voz e verificação gráfica explícita.

Integrações Especializadas: Roblox Studio e Blender

O suporte do assistente a ambientes de criação e desenvolvimento 3D utiliza protocolos oficiais de integração para garantir estabilidade e longevidade do código.

Roblox Studio: Servidor MCP Oficial Embutido vs Soluções Comunitárias

A análise das opções de integração com o Roblox Studio demonstrou uma evolução recente significativa na plataforma. No final de 2024, a Roblox Corporation incorporou um **servidor Model Context Protocol (MCP) nativo e embutido diretamente no executável do Roblox Studio**.

1. **Servidor MCP Nativo Embutido**: É ativado diretamente nas configurações do assistente interno do Roblox Studio (Gerenciador de Servidores MCP). Ele expõe uma interface de transporte STDIO por meio da execução do script binário %LOCALAPPDATA%\Roblox\mcp.bat no Windows. O servidor nativo fornece ferramentas diretas como generate_procedural_model, insert_asset, inspect_instance, e gerenciamento de subagentes autônomos para exploração da árvore de instâncias (DataModel) e testes de jogo (playtest).
2. **Repositório de Referência Descontinuado (studio-rust-mcp-server)**: O repositório em Rust mantido anteriormente pela Roblox no GitHub foi formalmente descontinuado e arquivado, com recomendação expressa da fabricante para migração integral para o servidor MCP embutido no aplicativo.
3. **Servidores Comunitários de Terceiros**: Implementações comunitárias (como roblox-studio-opencode-mcp ou roblex-studio-mcp-server) dependem de um plugin em

Lua rodando no Studio que realiza requisições de *long polling* HTTP contra um servidor intermediário Node.js local (geralmente escutando na porta 28821).

4. **Decisão Arquitetural**: O Projeto Noite 2.0 adotará **exclusivamente o servidor MCP nativo embutido do Roblox Studio** via transporte STDIO. Essa decisão elimina a necessidade de executar ambientes de execução adicionais (como Node.js ou Rust) e garante suporte imediato às atualizações da API da Roblox.

Blender: Servidor de Sockets Integrado via API Python Nativa

A integração com o Blender explora o interpretador Python completo mantido nativamente pela aplicação (bpy).
A Noite 2.0 utilizará um Add-on customizado leve em Python instalado dentro do Blender. Ao ser ativado, o Add-on inicializa um servidor de WebSocket assíncrono escutando na interface local 127.0.0.1:8765. Quando o assistente precisa manipular objetos 3D, materiais ou animações, ele envia blocos de código Python estruturados para o servidor do Add-on. O Add-on agenda a execução do código dentro da thread principal do Blender por meio do utilitário bpy.app.timers, captura os retornos do console e devolve o resultado formatado em JSON para o assistente.

Comparativo de Frameworks de Interface Gráfica (GUI)

Para substituir a interface obsoleta em Tkinter do sistema antigo, foram avaliadas três tecnologias modernas para desenvolvimento de aplicações Desktop em termos de footprint de recursos e facilidade de integração.

Critério de Análise	PySide6 (Qt6 para Python)	Tauri 2.0 (Rust + Web Frontend)	Electron (Node.js + Chromium)	Tkinter (Legado)
Consumo de Memória RAM	Baixo (~45 MB em idle)	Mínimo (~25 MB em idle)	Altíssimo (> 220 MB em idle)	Mínimo (~15 MB)
Tempo de Inicialização	Rápido (< 300 ms)	Instantâneo (< 100 ms)	Lento (> 1.200 ms)	Instantâneo
Integração com o Backend	Nativa (Mesmo processo Python)	Inter-Processos (IPC / WebSockets)	Inter-Processos (IPC / Sockets)	Nativa
Flexibilidade de Design	Alta (QML e Estilos Qt)	Máxima (React, Vue, HTML/CSS)	Máxima (HTML/CSS)	PÉSSIMA (Aparência anos 90)
Complexidade de Buildup	Simples (pip install pyside6)	Alta (Exige toolchain de Rust)	Média (Exige ecossistema npm)	Zero (Incluso no Python)
Decisão de Arquitetura	**Escolhido para o MVP**	**Avaliar para o Longo Prazo**	**Descartado Definitivamente**	**Descartado Definitivamente**

A escolha do **PySide6 (Qt6)** para a fase MVP do Projeto Noite 2.0 fundamenta-se na capacidade de construir janelas personalizadas sem bordas (*frameless windows*), componentes transparentes, animações fluidas e suporte nativo a temas escuros sem abandonar o ecossistema Python. O PySide6 roda diretamente no mesmo ambiente do núcleo assíncrono (asyncio), eliminando a necessidade de compilar pontes IPC complexas entre linguagens

diferentes no início do desenvolvimento.

Privacidade, Observabilidade e Suíte de Avaliação (Evals)

A governança operacional do assistente é mantida por meio de registros estruturados, mecanismos de proteção de dados pessoais e rotinas de avaliação contínua de regressão.

Observabilidade e Logs Estruturados

Em substituição aos logs em texto plano desestruturados da versão antiga, a Noite 2.0 utiliza a biblioteca **OpenTelemetry** configurada para gravar métricas em um banco de dados SQLite dedicado (telemetry.db). Toda interação do usuário gera um identificador único de rastreamento (trace_id) que correlaciona:

- O modelo exato consultado e os parâmetros de temperatura utilizados.
- A latência até o primeiro token de resposta (TTFT) e o tempo total de inferência.
- O número preciso de tokens de entrada e saída consumidos.
- O custo financeiro estimado da requisição em frações de dólar.
- Os logs de execução das ferramentas e códigos chamados.

Privacidade e Tratamento de Dados Pessoais

Antes de qualquer contexto ser enviado a provedores de inteligência artificial na nuvem (como Anthropic, OpenAI ou Google), o texto passa obrigatoriamente por uma camada local de sanitização de privacidade baseada em expressões regulares e na biblioteca Spacy. Este filtro identifica e oculta automaticamente informações sensíveis, substituindo números de CPF, senhas, chaves privadas, endereços IP locais e credenciais por tokens genéricos (por exemplo, <REDACTED_I[span_52](start_span)[span_52](end_span)[span_149](start_span)[span_149](end_span)P>).
Além disso, a interface disponibiliza um seletor visual denominado "Modo Estritamente Local" (Off-Grid). Quando ativado, o Gateway de modelos bloqueia fisicamente qualquer requisição de saída para a rede externa, forçando o assistente a operar exclusivamente com o modelo local Qwen 2.5 3B e ferramentas locais.

Suíte de Avaliação Automática (Evals Framework)

Para evitar degradação de desempenho ou perdas de comportamento ao alterar prompts ou substituir modelos locais, o projeto estabelece uma suíte de avaliação com 50 cenários de teste automatizados executados via pytest:

- **Avaliação do Roteador**: Garante que os comandos determinísticos sejam direcionados com 100% de acurácia sem disparar chamadas a modelos em nuvem.
- **Conformidade de JSON Schema**: Valida se o modelo local Qwen 2.5 3B mantém índice de acerto de 100% na geração de chamadas de funções estruturadas.
- **Resistência a Jailbreaks**: Submete o assistente a tentativas deliberadas de injeção de prompt e verifica se a persona e as restrições de segurança do Windows permanecem intactas.

Síntese Crítica das Propostas do Claude

A contribuição técnica fornecida pelo Claude na fase de conceituação inicial foi submetida a um processo de verificação confrontando suas hipóteses com documentações oficiais e testes em hardware real.
| Ponto Crítico Levantado pelo Claude | Posição Original do Claude | Análise e Avaliação Fundamentada | Veredito Arquitetural Adotado | | :--- | :--- | :--- | :--- | | **Gargalo de VRAM em Hardware 6 GB** | Afirmou ser impossível rodar LLM, STT e TTS juntos na GPU. | **Confirmado**: A VRAM de 6 GB estoura se todos usarem a GPU. | **Adotado**: LLM alocado na GPU (CUDA); STT/TTS alocados na CPU. | | **Incompatibilidade da CPU i7-3770** | Afirmou que a falta de AVX2 causaria falha crítica. | **Ajustado**: O Ollama 0.5.8+ roda via GPU sem exigir AVX2 da CPU. | **Ajustado**: Uso da GPU para LLM invalida o gargalo de AVX2 da CPU. | | **Integração Inicial de 4 Modelos** | Criticou integrar 4 modelos no primeiro dia. | **Confirmado**: Aumenta exponencialmente a complexidade. | **Adotado**: MVP inicia com apenas 1 local (Qwen 3B) e 1 nuvem (Claude 3.5). | | **Roteamento Baseado em Regras** | Recomendou regras determinísticas em vez de LLM. | **Confirmado**: Reduz latência de 1.5s para < 5ms em tarefas simples. | **Adotado**: Roteador Hierárquico em 3 Camadas implementado. | | **Adoção do SQLite + sqlite-vec** | Sugeriu substituir ChromaDB por SQLite vetorial. | **Confirmado**: Consumo de RAM cai de ~800 MB para < 10 MB. | **Adotado**: Extensão sqlite-vec escolhida como banco padrão. | | **Integração com Roblox Studio** | Sugeriu investigar servidores comunitários MCP. | **Refutado / Ajustado**: O Roblox Studio lançou MCP oficial embutido. | **Ajustado**: Adotado o servidor MCP nativo oficial do Roblox Studio. | | **Prioridade de Desenvolvimento** | Recomendou priorizar o núcleo por texto antes da voz. | **Confirmado**: Garante uma arquitetura estável e testável. | **Adotado**: Construção em fases focando primeiro no núcleo CLI. |

Critérios de Decisão e Definição da Arquitetura (MVP vs. Longo Prazo)

Para assegurar uma implementação sustentável, todos os componentes recomendados foram submetidos a critérios rigorosos de decisão técnica.

Matriz de Decisão por Componente Recomendado

Componente Escolhido	Problema Específico que Resolve	Principais Alternativas Consideradas	Limitações do Componente	Funciona no Hardware Atual?	Custo Recorrente	Impacto de Substituição Futura
LiteLLM	Padroniza a comunicação com múltiplos provedores de IA.	LangChain, SDKs nativos dos fornecedores.	Pequena sobrecarga de código de abstração.	Sim (Consumo nulo de recursos).	Gratuito (Código aberto).	Mínimo (Interface padrão OpenAI).
Qwen 2.5 3B (Q4_K_M)	Oferece capacidade	Llama 3.2 3B, Phi-3	Capacidade limitada em	Sim (Consome	Gratuito (Código	Mínimo (Gerenciado

Componente Escolhido	Problema Específico que Resolve	Principais Alternativas Considerada s	Limitações do Componente	Funciona no Hardware Atual?	Custo Recorrente	Impacto de Substituição Futura
	de raciocínio local veloz e privado.	Mini, Gemma 2 2B.	raciocínios matemáticos complexos.	2,2 GB de VRAM).	aberto).	via Ollama).
SQLite + sqlite-vec	Gerencia dados estruturados, histórico e busca vetorial.	ChromaDB, Qdrant, DuckDB, PostgreSQL.	Não possui suporte a índices grafos avançados (ANN HNSW).	Sim (Consumo < 10 MB de RAM).	Gratuito (Licença MIT).	Médio (Exige exportação de dados via SQL).
faster-whisp er (Small)	Transcreve fala em texto com precisão em português.	Vosk PT-BR, OpenAI Whisper API, Deepgram.	Transcrição mais lenta na CPU por ausência de AVX2.	Sim (Consome 250 MB de RAM na CPU).	Gratuito (Licença MIT).	Baixo (Isolado no serviço de áudio).
Piper TTS Engine	Sintetiza voz natural de forma totalmente local.	Microsoft Edge-TTS, ElevenLabs API, pyttsx3.	Quantidade restrita de vozes neutras em português.	Sim (Consome 120 MB de RAM na CPU).	Gratuito (Licença MIT).	Baixo (Isolado no serviço de áudio).
PySide6 (Qt6)	Expõe interface gráfica Desktop moderna e fluida.	Tkinter, Tauri 2.0, Electron.	Curva de aprendizado superior ao Tkinter.	Sim (Consome ~45 MB de RAM).	Gratuito (Licença LGPL).	Médio (Reescrita da camada de visão).
Roblox Studio MCP	Permite manipulação programática do Roblox Studio.	st[span_218](start_span)[span_218](end_span)[span_225](start_span)[span_225](end_span)udio-rust-mcp-server, plugins HTTP.	Requer a aplicação do Roblox Studio em execução.	Sim (Executado via script nativo mcp.bat).	Gratuito (Oficial da plataforma).	Baixo (Baseado no padrão aberto MCP).

Escopo da Arquitetura MVP vs. Longo Prazo

Estágio MVP (Versão Mínima Viável - Fases 1 a 4)

- **Interface**: Linha de comando avançada (CLI) e caixa de texto simples flutuante em PySide6.

- **Modelos Integrados**: Modelo local Qwen 2.5 3B (via Ollama) + Anthropic Claude 3.5 Sonnet na nuvem (via LiteLLM).
- **Roteamento**: Roteador Determinístico por Regras (Regex) + Classificador Local SLM.
- **Memória**: SQLite relacional básico com extensão FTS5 para busca em histórico de texto.
- **Ferramentas de Controle**: Leitura e escrita restrita na pasta de trabalho /workspace/ e controle básico de janelas via pywinau[span_89](start_span)[span_89](end_span)[span_186](start_span)[span_186](end_span)to.

Estágio Longo Prazo (Versão 2.0 Completa - Fases 5 a 8)

- **Interface**: Dashboard completo em PySide6 com indicadores de status de agentes, linha do tempo de tarefas e visualizador de estado de memória.
- **Modelos Integrados**: Suporte dinâmico aos 4 provedores (OpenAI, Anthropic, Google e Ollama) com controle automático de custos por tarefa.
- **Memória**: Sistema completo SQLite + sqlite-vec com resumos de episódios e busca vetorial densa.
- **Voz**: Pipeline local contínuo (Silero VAD + openWakeWord + faster-whisper + Piper TTS) com suporte a interrupção por fala (*barge-in*).
- **Especializações**: Integração total com Roblox Studio (via MCP oficial embutido) e Blender (via Add-on WebSocket).

Suíte de Testes no Hardware e Checklist de Aprovação

Antes do início da codificação da Noite 2.0, as verificações práticas a seguir devem ser executadas no computador do usuário para homologação das métricas de desempenho do hardware real.

Suíte de Testes Práticos de Validação Local

1. **Teste 1: Aceleração CUDA no Ollama sem AVX2**
    - *Objetivo*: Validar se a inferência do modelo local é executada integralmente na GPU sem dependência do conjunto de instruções AVX2 da CPU.
    - *Procedimento no Terminal*:
      ollama run qwen2.5:3b "Escreva um algoritmo de ordenacao em Python." --verbose `[span_322](start_span)[span_322](end_span)``

    - *Critério de Sucesso*: Os logs de execução do Ollama devem confirmar que 100% das camadas foram carregadas na VRAM (offloaded 35/35 layers to GPU) e a taxa de geração deve atingir no mínimo **40 tokens por segundo**.
2. **Teste 2: Monitoramento de Consumo Limite de VRAM**
    - *Objetivo*: Verificar a folga de memória de vídeo durante sessões de conversa com contexto longo.
    - *Procedimento no PowerShell*:
      nvidia-smi --query-gpu=memory.used,memory.free --format=csv -l 1

    - *Critério de Sucesso*: O consumo total de VRAM da placa de vídeo não deve ultrapassar **4,0 GB** durante o processamento de uma janela de contexto de 4.096 tokens.
3. **Teste 3: Latência de Transcrição no faster-whisper via CPU**
    - *Objetivo*: Quantificar a velocidade de processamento do modelo Whisper na CPU i7-3770 sem instruções AVX2.
    - *Procedimento em Código Python*:
      import time
      from faster_whisp[span_244](start_span)[span_244](end_span)er import WhisperModel

      model = WhisperModel("small", device="cpu", compute_type="int8")
      start_time = time.time()
      segments, _ = model.transcribe("audio_teste_5s.wav")
      texto_transcrito = "".join([segment.text for segment in segments])
      print(f"Tempo decorrido: {time.time() - start_time:.2f}s | Resultado: {texto_transcrito}")

    - *Critério de Sucesso*: A transcrição de um arquivo de áudio de 5 segundos de duração deve ser concluída em um tempo total inferior a **1,2 segundo**.
4. **Teste 4: Carregamento da Extensão C sqlite-vec no Windows**
    - *Objetivo*: Garantir que o Python local carrega corretamente a DLL pré-compilada do sqli[span_213](start_span)[span_213](end_span)te-vec no ambiente Windows.
    - *Procedimento em Código Python*:
      import sqlite3

import sqlite_vecconexao = sqlite3.connect(":memory:") conexao.enable_load_extension(True) sqlite_vec.load(conexao) conexao.enable_load_extension(False) versao = conexao.execute("SELECT vec_version()").fetchone()[0] print(f"Extensão sqlite-vec carregada com sucesso. Versão: {versao}")* *Critério de Sucesso*: O script deve imprimir a versão `v0.1.9` no console sem disparar exceções de biblioteca vinculada ou DLL não encontrada[span_346](start_span)[span_346](end_span).

Decisões Submetidas à Aprovação do Diretor do Projeto

As diretrizes técnicas listadas abaixo necessitam de aprovação formal para o início do desenvolvimento:

- **Decisão 1: Descarte da Interface Legada e Substituição por PySide6**: Aprovação para descartar o arquivo gui.py em Tkinter e construir uma nova interface gráfica desacoplada em PySide6 (Qt6).
    - [ ] Aprovado [ ] Rejeitado
- **Decisão 2: Roteamento Hierárquico em Três Camadas**: Aprovação para adotar o pipeline de decisão baseado primeiramente em Regras Determinísticas (Regex), seguido pelo Modelo Local Pequeno (SLM Qwen 3B) e escalando para APIs em Nuvem apenas em tarefas complexas.
    - [ ] Aprovado [ ] Rejeitado

- **Decisão 3: Adoção do SQLite com a Extensão sqlite-vec**: Aprovação para consolidar a persistência de histórico, fatos e vetores dentro do banco de dados leve SQLite (sqlite-vec), dispensando bancos vetoriais dedicados como ChromaDB.
    - [ ] Aprovado [ ] Rejeitado
- **Decisão 4: Substituição do Pipeline de Voz por Soluções 100% Locais**: Aprovação para eliminar o uso das bibliotecas edge-tts e playsound, adotando a pilha local composta por Silero VAD, openWakeWord, faster-whisper e Piper TTS com suporte a interrupções (*barge-in*).
    - [ ] Aprovado [ ] Rejeitado
- **Decisão 5: Integração com o Roblox Studio via MCP Oficial Embutido**: Aprovação para utilizar exclusivamente a interface nativa MCP do Roblox Studio (%LOCALAPPDATA%\Roblox\mcp.bat), eliminando dependências de servidores comunitários intermediários.
    - [ ] Aprovado [ ] Rejeitado
- **Decisão 6: Modelo de Permissões e Segurança do Windows**: Aprovação para impor a execução do assistente sob conta de usuário sem elevação administrativa, vinculação de processos filhos a Win32 Job Objects e gestão de chaves no Windows Credential Manager.
    - [ ] Aprovado [ ] Rejeitado

Referências citadas

1. Projeto Noite 2.0 — Prompt completo de pesquisa para o Gemini.pdf, https://drive.google.com/open?id=1-bgW8xXB5gqpT-FD1umM09kB-CnQWpbr 2. Projeto Noite 2.0 — Prompt completo de pesquisa para o Gemini, https://drive.google.com/open?id=126mcMksmVZXuQo1LfTk-AcCXMYeeDxXcB6viRTfzFxI 3. error: illegal instruction on CPUs without AVX or AVX2 ... - GitHub, https://github.com/ollama/ollama/issues/644 4. Can it support avx cpu's older than 10 years old? #451 - GitHub, https://github.com/ggml-org/llama.cpp/issues/451 5. Ollama 0.5.8 adds AVX-512 CPU acceleration and AVX2 for NVIDIA, https://www.reddit.com/r/LocalLLaMA/comments/1ijyrlv/ollama_058_adds_avx512_cpu_acceleration_and_avx2/ 6. Run llama.cpp for Local Inference: Setup Guide 2026, https://khimananda.com/blog/run-llama-cpp-for-local-inference 7. Llama.cpp - non-AVX processors? : r/LocalLLaMA - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1muwsj6/llamacpp_nonavx_processors/ 8. AI Models for GTX 1660 Super 6GB, https://willitrunai.com/gpus/gtx-1660-super-6gb 9. Faster Whisper transcription with CTranslate2 - GitHub, https://github.com/SYSTRAN/faster-whisper 10. Choosing between Whisper variants: faster-whisper, insanely-fast, https://modal.com/blog/choosing-whisper-variants 11. A GPU-Poor's Guide to Local LLM Inference in 2026 - Towards AI, https://pub.towardsai.net/a-gpu-poors-guide-to-local-llm-inference-in-2026-48d59cafd215 12. Pydantic AI Review 2026 | AI Infrastructure & MLOps Tool, https://aiagentslist.com/agents/pydantic-ai 13. Pydantic AI: Features, Pricing, Pros & Cons (2026) - AI IDE List, https://aiidelist.com/ide/pydantic-ai 14. Pydantic AI V2 in Late 2026: Capabilities, Graphs, Evals and MCP, https://www.developersdigest.tech/blog/pydantic-ai-whats-new-2026 15. sqlite-vec - PyPI, https://pypi.org/project/sqlite-vec/ 16. GitHub - asg017/sqlite-vec: A vector search SQLite extension that, https://github.com/asg017/sqlite-vec 17. Using sqlite-vec in Python - Alex Garcia,

https://alexgarcia.xyz/sqlite-vec/python.html 18. sqlite-vec download | SourceForge.net, https://sourceforge.net/projects/sqlite-vec.mirror/ 19. Introducing sqlite-vec v0.1.0: a vector search SQLite extension that, https://www.reddit.com/r/LocalLLaMA/comments/1ehlazq/introducing_sqlitevec_v010_a_vector_search_sqlite/ 20. Connect to the Roblox Studio MCP server | Documentation, https://create.roblox.com/docs/studio/mcp 21. GitHub - Roblox/studio-rust-mcp-server, https://github.com/Roblox/studio-rust-mcp-server 22. GitHub - Z4H1DD/RobloxMCP, https://github.com/Z4H1DD/RobloxMCP 23. dmae97/roblex-studio-mcp-server: Model Context Protocol ... - GitHub, https://github.com/dmae97/roblex-studio-mcp-server 24. paralov/roblox-studio-opencode-mcp - GitHub, https://github.com/paralov/roblox-studio-opencode-mcp